

Coding Assignment

Course: Distributed Systems

General Instructions

- Attempt **allotted** questions in this homework.
- **Cluster Usage:**
 - All implementations must run on the provided CPU cluster.
 - Each team will be **soon mailed cluster credentials**.
 - Assume availability of up to **24 cores per node** for programs (scalability tests required).
- Deliverables:
 - (a) Source code files.
 - (b) A short README (1–2 pages) with compilation and execution instructions.
 - (c) A detailed report (see Section 4).
- Submission format:
 - All deliverables must be packed into a single compressed archive (**.zip** or **.tar.gz**).
 - Filename: **RollNo_Homework2**.
 - Submit via the Moodle submission portal. (portal for the same will be made soon)
 - Same submission will be downloaded and ran during viva
- Deadline: **[9 September]**.

1 MPI Programming Questions

1.1 Sparse Matrix Multiplication (Row-Row Format)

- Implement sparse matrix multiplication using the row-row format.
- Distribute rows across MPI processes and perform parallel multiplication.
- Ensure that intermediate communication is minimized.

Constraints

- Matrix dimensions:

$$1 \leq N, M, P \leq 10001$$

- Number of non-zero elements per matrix:

$$\leq 10^5$$

- Input matrices are guaranteed to be **sparse**.
- **Important:** You must *not* convert the matrices into dense format during multiplication.

Input Format

- First line: Three integers $N, M, P \rightarrow$ dimensions of matrices $A \in R^{N \times M}$ and $B \in R^{M \times P}$.
- Next N lines: Sparse row representation of matrix A . Each row has the format:

$$k \text{ } col_1 \text{ } val_1 \text{ } col_2 \text{ } val_2 \text{ } \dots \text{ } col_k \text{ } val_k$$

where:

- k = number of non-zero elements in that row
 - col_j = column index (0-based)
 - val_j = value at (row, col_j)
- Next M lines: Sparse row representation of matrix B (same format as above).

Output Format

- Print the result matrix C of size $N \times P$ in sparse row format.
- Each line corresponds to one row of C .
- Format:

$$k \text{ } col_1 \text{ } val_1 \text{ } col_2 \text{ } val_2 \text{ } \dots \text{ } col_k \text{ } val_k$$

Sample Input

```
3 3 3
2 0 1 2 2
1 1 3
0
1 0 2
2 0 4 2 5
1 1 6
```

Sample Output

```
2 0 2 1 12
1 0 12
0
```

1.2 Matrix Inversion using Gauss–Jordan Elimination

- Given a non-singular square matrix A of size $n \times n$, compute its inverse using the Gauss–Jordan elimination method.
- Workload distribution: Each process handles a subset of rows during elimination steps.
- **Input Format:**

```

N
A[0][0] A[0][1] ... A[0][N-1]
...
A[N-1][0] ... A[N-1][N-1]
```

- **Output Format:** Print N lines of N floating-point numbers (exactly 2 decimal places) representing A^{-1} .
- Constraints:
 - $2 \leq N \leq 1000$.
 - Matrix is guaranteed to be non-singular.
 - $-1000.00 \leq A[i][j] \leq 1000.00$.

- Example:

Input:

```
3
4.00 7.00 2.00
3.00 6.00 1.00
2.00 5.00 3.00
```

Output:

```
1.44 -1.22 -0.56
-0.78 0.89 0.22
0.33 -0.67 0.33
```

1.3 Matrix Chain Multiplication using Parallel DP

- Given N matrices $A_1, A_2, \dots, A_N$ with dimensions $d_0 \times d_1, d_1 \times d_2, \dots, d_{N-1} \times d_N$, determine the optimal parenthesization that minimizes the number of scalar multiplications.
- Implement parallel dynamic programming using MPI:
 - Distribute computation of DP table entries across processes.
 - Each process handles a subset of computations for the cost table.

- **Input Format:**

```
N
d0 d1 d2 ... dN
```

- **Output Format:** A single integer representing the minimum scalar multiplications required.
- Constraints: $1 \leq N \leq 500, 1 \leq d_i \leq 1000$.
- Example:

```
Input:
4
1 2 3 4 3
```

```
Output:
30
```

1.4 Longest Common Subsequence (Parallel DP)

- Given two strings A and B , compute their Longest Common Subsequence (LCS) using parallel dynamic programming with MPI.
- Distribute rows/columns of the DP matrix across processes.
- **Input Format:**

```
A
B
```

- **Output Format:** A string representing the LCS.
- Constraints: $1 \leq |A|, |B| \leq 10000$.
- Example:

Input :
ABCBADAB
BDCABB

Output :
BCAB

2 MapReduce Programming Questions

2.1 K-Nearest Neighbors (K-NN)

- Implement distributed K-NN search using MapReduce.
- Input: `points.csv`, `queries.csv`, parameter K .
- CLI:

```
./run_knn_mapreduce.sh points.csv queries.csv K output_dir
```

- Output: `results.txt`, each line contains K nearest neighbors for a query point.
- Evaluation: Verify correctness and analyze performance under different cluster sizes.

2.2 K-Means Clustering

- Implement standard iterative K-Means clustering with MapReduce.
- Mapper: Assigns points to nearest center.
- Reducer: Computes new centers.
- Stop when convergence is reached or T iterations complete.
- Output: Final centers and point-to-cluster assignment file.
- CLI:

```
./run_mapreduce.sh K points.csv centers.txt max_iter output_dir
```

2.3 Triangle Counting in a Graph

- Count total number of triangles and per-vertex participation counts.
- Input: Edge list file (`u v`, with $u < v$).
- Output:
 - Global count of triangles.
 - Per-vertex participation counts.
- Ensure no double-counting (orient edges, degree-based ordering).

2.4 4-Cycle Counting in a Graph

- Count simple 4-cycles in an undirected graph.
- Input: Edge list $(u, v, u < v)$.
- Output:
 - Global count of 4-cycles.
 - Per-vertex participation counts.
- Deduplication: use canonical ordering or combinatorial factors to avoid overcounting.

3 gRPC Programming Questions

3.1 Monitoring Service

- Implement a gRPC service to monitor a machine.
- RPCs:
 - (i) `ListUsers()` – returns list of logged-in users.
 - (ii) `ListJobs()` – returns a stream of jobs (streaming RPC).
 - (iii) `GetLongestRunningJob()` – returns job with maximum runtime.
- Input: Simulated via text files.
- Output: JSON or text format.
- Support multiple concurrent clients.

3.2 Graph Queries with Two Clients

- Two clients submit graphs asynchronously; server maintains their union.
- RPCs:
 - (i) `HasIndependentSet(k)`
 - (ii) `HasMatching(k)`
- Input: Adjacency list.
- Output: Boolean query results.

3.3 Matrix Queries with Two Clients

- Two clients submit matrix rows asynchronously.
- Server forms combined matrix and answers:
 - (i) `HasRankAtLeast(r)`
 - (ii) `HasDeterminantAtLeast(d)`
- Input: Each client contributes one row at a time.
- Output: Boolean results.

3.4 Range Tree Service

- Client A streams 2D points to the server.
- Client B queries with rectangles.
- Server builds a range tree and supports:
 - (i) `CountPointsInRectangle(rect)`
 - (ii) `GetTopPoint(rect)`
- Constraint: Server rebuilds tree when Client A updates data; queries blocked during rebuild.

4 Report Guidelines

Along with source code, each student must submit a **report (6–8 pages)** in PDF (generated via LaTeX). The report must include:

1. Problem Understanding: Clear description of the problem and approach.

2. System Setup:

- Cluster configuration (number of cores, nodes, memory).
- MPI/MapReduce/gRPC setup details.

3. Design and Implementation:

- Parallelization strategy.
- Data distribution approach.
- Communication patterns.

4. Execution Details:

- Exact command lines used.

5. Performance Evaluation: Graphs and visualization are recommended

- Execution time vs. number of cores.
- Speedup and efficiency plots.
- Scalability analysis.

6. Challenges and Limitations: Discuss difficulties faced, bottlenecks observed.

7. Conclusion and Future Work.

Reports will be evaluated on **clarity, completeness, and depth of performance analysis.**